Blast Radius

Learning walkthrough: architecture, graph model, every query, and the questions you will be asked

Graph Hacks 2026 (WeMakeDevs x FalkorDB) | github.com/kushalkumarcs372/blast-radius | blast-radius-duqf.onrender.com

How to use this guide

Read sections 1 to 4 first; they are what every judge and interviewer asks about. Then run each query yourself in FalkorDB Browser (localhost:3000, graph *tenant_acme*) while reading section 4. Finish with the question bank in section 10 and answer each question out loud before reading the answer.

1. The project in 60 seconds

Problem. When an alert fires at 2 AM, the on-call engineer needs four answers fast: what else is affected, what caused it, who should fix it, and has this happened before. That information is spread across dependency maps, deploy logs, ownership pages and old postmortems. Searching documents one at a time misses how those things connect.

Solution. Blast Radius stores a company's services, dependencies, deploys, incidents, people, runbooks and design decisions in a FalkorDB graph. Three agents work an alert in order: **Triage** finds what is at risk and what changed, **Investigator** compares with past incidents and picks the likely cause with a confidence, **Responder** finds who should act and proposes the fix. They pass findings to each other by writing *Episode* nodes into the graph, so the work is remembered across sessions. Every answer shows the graph path and the node IDs it came from.

One-line pitch: *An on-call agent team whose every answer is a path you can see in the graph.*

The demo result: an alert on *postgres-payments* puts 8 services at risk (4 customer-facing). The only change in the 6 hours before is a payments deploy that raised the connection pool from 50 to 400, 38 minutes earlier. The same change caused an incident 3 weeks ago, which Sam Costa fixed in 42 minutes, so the confidence is 0.96. The proposed action: roll back, page Sam (on call), follow the runbook. The whole run takes about 3 seconds.

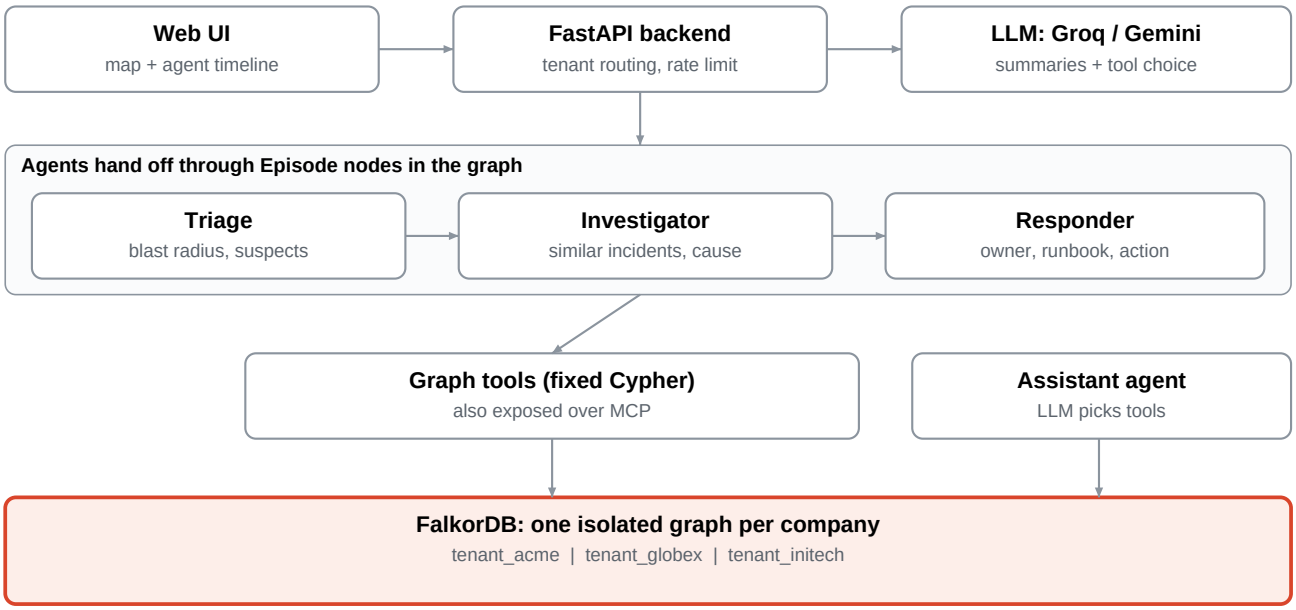
Why this needs a graph

Each question is about connections, not single records:

Question	What the graph does	Without a graph
What breaks if X goes down?	One variable-length path query, 1 to 4 hops	Repeated joins or code that loops over a dependency table
What changed near X?	Walk 2 hops either way, then to Deployment nodes	Several queries stitched together in application code
Has this happened before?	Incident -> Service, Incident -> Deployment, Person -> Incident in one pattern	Searching postmortem text and hoping the words match
Who knows this system?	Person -[:RESOLVED]-> Incident -[:AFFECTED]-> Service, counted	Asking around in chat

Hackathon rule: if the product works with FalkorDB removed, the graph is decoration. Here every agent step is a graph query and agent memory lives in the graph.

2. Architecture



Component	File	What it does
Graph tools	blast_radius/tools/graph_tools.py	11 fixed Cypher queries. Each returns rows, the graph paths behind them (evidence) and the Cypher text.
Agents	blast_radius/agents/	triage.py, investigator.py, responder.py run in order (orchestrator.py). ask.py answers free-form questions.
Memory	blast_radius/memory.py	Creates incidents, writes Episode nodes and their links, reads them back for handoffs and later sessions.
LLM client	blast_radius/llm.py	Groq first, Gemini as fallback (configurable). Retries 503s; falls back on any error.
API + UI	blast_radius/api.py, web/index.html	FastAPI endpoints; a single-page UI that draws the service map with Cytoscape.js.
MCP server	blast_radius/mcp_server.py	Exposes the same graph tools to any MCP client such as Claude Desktop.
Data	blast_radius/seed/	catalog.py (hand-written architecture), generator.py (deterministic synthetic company), loader.py (batched UNWIND).
Tests	tests/	61 tests: data checks, every query against an independent Python answer, agents with a fake LLM, the HTTP API.

Key design decision: the graph decides, the LLM explains

Which services are at risk, which deploy is suspected, the confidence score and who to page all come from Cypher and plain Python. The LLM only writes the human-readable summary, and for follow-up questions it chooses which fixed tool to call. It never writes Cypher. Three reasons:

- **Correctness:** the facts are testable. The tests check them without any LLM.
- **Free tiers:** if both LLM providers are rate limited, the agents still finish with template summaries.
- **Safety:** a model cannot run a destructive query, because it can only pick from a fixed tool list.

Hallucination guard (agents/base.py): every LLM summary must cite node IDs in square brackets. If it cites an ID that is not in the facts it was given, the summary is rejected and the template is used instead.

The assistant agent flags invented IDs in its answer. You saw this work live when the model invented *[owners-001]*.

Request flow when you click Run agents

```
1. POST /api/respond {alert: "alert-demo-live", tenant: "acme"}
2. Triage:       open_incident() MERGEs Incident node, runs blast_radius, recent_deploys,
                 critical_services; writes AT_RISK edges + Episode (cites alert, services, deploys)
                 -> HANDED_OFF_TO investigator
3. Investigator: reads Triage's Episode FROM THE GRAPH, runs similar_incidents per suspect,
                 scores confidence; writes SUSPECTED_CAUSE and SIMILAR_TO edges + Episode
                 -> HANDED_OFF_TO responder
4. Responder:    reads Investigator's Episode, runs owners, fix_history, runbook,
                 decision_trace; builds actions; sets status; writes Episode
5. UI:           GET /api/incidents/inc-alert-demo-live -> draws paths, timeline, actions
```

3. The graph data model

Each company is a separate named graph in one FalkorDB instance: *tenant_acme*, *tenant_globex*, *tenant_initech*. A query on one graph can never see another's nodes. This is how the project handles multi-tenancy, and FalkorDB is built for thousands of such graphs in one instance.

Nodes

Label	Key properties	Count in acme
Service	id, name, tier (edge/core/data/platform/external), language	38
Team	id, name, channel	9
Person	id, name, role, on_call	about 30
Deployment	id, version, ts (epoch ms), summary, service	152
Alert	id, signal, severity, ts, service	27
Incident	id, title, severity, status, started, resolved	26 (+ live ones)
Runbook	id, title, steps	one per core/data service
Decision / Document	title, summary, date / title, type, source	6 ADRs + postmortems
Agent / Episode	name, role / id, agent, session, kind, ts, summary, findings (JSON), confidence	4 agents; grows

Relationships

```
(Person)-[:MEMBER_OF]->(Team)-[:OWNS]->(Service)
(Service)-[:DEPENDS_ON]->(Service)                54 edges; direction = "needs"
(Person)-[:AUTHORED]->(Deployment)-[:CHANGED]->(Service)
(Alert)-[:FIRED_ON]->(Service)
(Incident)-[:TRIGGERED_BY]->(Alert)
(Incident)-[:AFFECTED]->(Service)
(Incident)-[:CAUSED_BY]->(Deployment)
(Person)-[:RESOLVED]->(Incident)
(Runbook)-[:FOR]->(Service)
(Decision)-[:ABOUT]->(Service)
(Decision)-[:SOURCED_FROM]->(Document)
(Document)-[:MENTIONS]->(Incident)

Agent memory (written at runtime):
(Agent)-[:RECORDED]->(Episode)-[:ABOUT]->(Incident)
(Episode)-[:HANDED_OFF_TO]->(Agent)
(Episode)-[:CITES]->(Service | Deployment | Person | Incident | Runbook | Document | Alert)
(Incident)-[:AT_RISK {hops}]->(Service)
(Incident)-[:SUSPECTED_CAUSE {confidence}]->(Deployment)
```

```
(Incident)-[:SIMILAR_TO {score}]->(Incident)
```

Direction matters. *(checkout)-[:DEPENDS_ON]->(payments)* means checkout needs payments. So the *blast radius* of payments is everything with a path *into* it, and the *dependencies* of checkout are everything reachable *out of* it.

Indexes: *id* on every label, plus *Service.name*, *Team.name*, *Deployment.ts*, *Episode.id* and *Episode.session*. Every query starts from a service name, so the start node is an index lookup instead of a scan.

Memory types (Track 02 vocabulary): episodic = Episode nodes (what each agent found, when, in which session); semantic = the company graph itself; procedural = Runbook nodes plus SIMILAR_TO links to past fixes.

4. Every query, explained

Run each in FalkorDB Browser on *tenant_acme* with real values substituted. Being able to read these out loud is the most important preparation for judging.

blast_radius: what breaks if X goes down?

```
MATCH p = (up:Service)-[:DEPENDS_ON*1..4]->(s:Service {name: $service})
RETURN up.name, up.tier, length(p), [n IN nodes(p) | n.name]
```

Finds every service with a dependency path of 1 to 4 hops *into* the given service. **1..4* is a variable-length relationship; *nodes(p)* returns the path so the UI can draw it. Several paths can reach the same service; Python keeps the shortest one per service. The hop limit is validated as an integer from 1 to 6 before being put into the query, because Cypher cannot take it as a parameter. Demo answer: 8 services.

dependencies: what does X need?

```
MATCH p = (s:Service {name: $service})-[:DEPENDS_ON*1..4]->(d:Service)
RETURN d.name, d.tier, length(p), [n IN nodes(p) | n.name]
```

Same pattern, opposite direction.

recent_deploys: what changed near X just before the alert?

```
MATCH (a:Person)-[:AUTHORED]->(d:Deployment)-[:CHANGED]->(s:Service {name: $service})
WHERE d.ts >= $since AND d.ts <= $until
RETURN d.id, d.version, d.ts, d.summary, s.name, a.name, 0, [s.name]

MATCH p = (s:Service {name: $service})-[:DEPENDS_ON*1..2]-(n:Service)
MATCH (a:Person)-[:AUTHORED]->(d:Deployment)-[:CHANGED]->(n)
WHERE d.ts >= $since AND d.ts <= $until
RETURN d.id, d.version, d.ts, d.summary, n.name, a.name, length(p), [x IN nodes(p) | x.name]
```

The second query has no arrow on *DEPENDS_ON*1..2*, so it walks **both directions**. Why: a change to a *caller* can break the alerting service too. In the demo, the database did not change; *payments*, which depends on it, raised its connection pool and exhausted the database's connections. The window is the 6 hours *before the alert's timestamp*, not before now, so results stay correct when replayed later.

critical_services: which services matter most?

```
CALL algo.pageRank('Service', 'DEPENDS_ON') YIELD node, score
RETURN node.name, score ORDER BY score DESC LIMIT $limit
```

PageRank is a built-in FalkorDB algorithm. On DEPENDS_ON edges, score flows toward services that many others depend on, directly or indirectly, so shared infrastructure ranks highest (postgres-users, object-storage). A second query counts transitive dependents so the number is explainable. The UI sizes nodes by PageRank.

connection_path: how are A and B connected?

```
MATCH (a:Service {name: $a}), (b:Service {name: $b})
WITH shortestPath((a)-[:DEPENDS_ON*]->(b)) AS p
WHERE p IS NOT NULL
RETURN [n IN nodes(p) | n.name], length(p)
```

Tries A depends on B, then B depends on A, then any undirected link within 4 hops.

owners and fix_history: who acts, and who knows?

```
MATCH (t:Team)-[:OWNS]->(s:Service {name: $service})
OPTIONAL MATCH (p:Person)-[:MEMBER_OF]->(t)
RETURN t.name, t.channel, p.name, p.role, p.on_call, p.id

MATCH (p:Person)-[:RESOLVED]->(i:Incident)-[:AFFECTED]->(s:Service {name: $service})
OPTIONAL MATCH (p)-[:MEMBER_OF]->(t:Team)
RETURN p.name AS person, t.name, count(DISTINCT i) AS fixes, collect(DISTINCT i.id), p.id
ORDER BY fixes DESC, person
```

OPTIONAL MATCH keeps the team row even if it has no members. *fix_history* counts expertise from real incident history rather than from a job title.

similar_incidents: has this happened before?

```
MATCH (past:Incident)-[:AFFECTED]->(s:Service {name: $service})
WHERE past.status = 'resolved'
OPTIONAL MATCH (past)-[:TRIGGERED_BY]->(al:Alert)
OPTIONAL MATCH (past)-[:CAUSED_BY]->(d:Deployment)-[:CHANGED]->(ds:Service)
OPTIONAL MATCH (fixer:Person)-[:RESOLVED]->(past)
OPTIONAL MATCH (doc:Document)-[:MENTIONS]->(past)
RETURN past.id, past.title, past.started, past.resolved, al.signal,
       d.id, d.summary, ds.name, fixer.name, doc.title
```

One pattern collects the past incident's signal, cause, the service that cause changed, who fixed it and its postmortem. Python then scores each: **3** if the signal matches, **+2** if the old cause changed the same service as today's suspect, **+2 x word overlap** of the two change summaries. The demo match scores 6.6.

runbook, decision_trace, open_alerts

```
MATCH (r:Runbook)-[:FOR]->(s:Service {name: $service}) RETURN r.id, r.title, r.steps

MATCH (d:Decision)-[:ABOUT]->(about:Service)
MATCH (d)-[:SOURCED_FROM]->(doc:Document)
RETURN d.id, d.title, d.summary, doc.id, doc.title, doc.source, about.name, d.date AS date
ORDER BY date DESC          -- then kept if about = service OR the text names the service

MATCH (a:Alert)-[:FIRED_ON]->(s:Service)
OPTIONAL MATCH (i:Incident)-[:TRIGGERED_BY]->(a)
WITH a, s, i WHERE i IS NULL
RETURN a.id, a.signal, a.severity, a.ts, s.name ORDER BY a.ts DESC
```

decision_trace matches the service name as a whole word in Python so that *auth* does not match *authorisation*, which a plain CONTAINS would. *open_alerts* finds alerts with no incident yet using OPTIONAL MATCH plus *IS NULL*, the standard way to express "has no such relationship".

5. Agents, memory and the LLM

The three agents

Agent	Reads	Decides	Writes
Triage	The alert; blast_radius; recent_deploys (6 h before alert); critical_services	Services at risk, customer-facing ones, suspect deploys with minutes-before-alert	Incident (MERGE, idempotent), AT_RISK edges, Episode -> investigator
Investigator	Triage's Episode (from the graph); similar_incidents per suspect	Likely cause and confidence	SUSPECTED_CAUSE, SIMILAR_TO, Episode -> responder
Responder	Investigator's Episode; owners; fix_history; runbook; decision_trace	Rollback or review; who to page; expert to loop in; runbook; context	Status = mitigation_proposed, Episode
Assistant	Open incidents + any tool the LLM picks, one at a time (max 5 steps)	The answer, citing node IDs	Q&A; Episode in the session, CITES the nodes used

Confidence formula (investigator.py)

```
confidence = 0.35 x recency    (1 - minutes_before_alert / 360)
           + 0.25 x proximity  (1.0 if on the service or 1 hop away, else 0.5)
           + 0.40 x history    (min(1, best similar-incident score / 5))

Demo: recency 1 - 38/360 = 0.894, proximity 1.0, history min(1, 6.6/5) = 1.0
      0.35 x 0.894 + 0.25 + 0.40 = 0.96
```

Be ready to say it is a simple, explainable heuristic, not a trained model, and that with real data you would tune the weights against past incidents whose causes are known.

Why handoffs go through the graph

The Investigator does not receive Triage's output as a Python variable. It calls *latest_episode(incident, 'triage')*, which reads the Episode node. So a different process, a later session, the MCP server or the UI can all pick up exactly where the agents left off. That is the Track 02 claim: memory persists, is shared, and is available to every agent that needs it. Demonstrate it by clicking **Open** under Past sessions: the incident is rebuilt from the graph with no new LLM calls.

LLM layer

- **Providers:** Groq (openai/gpt-oss-120b) first, Gemini (gemini-flash-lite-latest) as fallback; order set by LLM_PROVIDER. Both free.
- **Fallback:** any error (429 rate limit, 503 overload, timeout, bad JSON) moves to the next provider; 503s are retried twice first. If all fail, templates are used.
- **JSON:** every call asks for a JSON object, so output can be validated. Groq's strict JSON mode is off because reasoning models fail its validator; the client extracts the JSON itself.
- **Assistant protocol:** the model replies either {"tool": name, "args": {...}} or {"answer": ...}. Unknown tools and bad arguments are rejected and the error is shown to the model, which then tries again.
- **Cost control:** /api/ask is limited to 30 questions per IP per hour on the public demo.

6. How we know it is correct

The tests never trust FalkorDB to grade itself. *seed/reference.py* computes each answer independently with a plain breadth-first search in Python over the same generated data. The tests load that data into FalkorDB, run each Cypher tool and assert the two answers are equal. If they agree on 5 different services, the query is doing what we claim.

Test file	What it proves
test_generator.py	Data is deterministic; no dependency cycles; every edge points at a real node; one owner per service; one on-call per team; no future timestamps; the demo scenario exists only in acme.
test_graph_tools.py	Every query matches the Python reference; tenants are isolated; bad input returns errors instead of crashing; decision_trace whole-word matching.
test_agents.py	Agents run in order with a scripted fake LLM; pick deploy-demo-suspect with confidence >= 0.8; handoff chain stored in the graph; memory readable from a new connection; assistant remembers the conversation and flags invented IDs.
test_llm.py	Provider fallback on 429/503/timeout; summaries citing unknown IDs are rejected.
test_api.py	HTTP endpoints end to end, including respond, incident view, reset and input validation.

Data is deterministic: the same tenant and clock always produce the same company, which is what makes the reference comparison possible. All data is synthetic, as the hackathon rules require.

7. Deployment and cost

Part	Where	Cost
API + UI	Render free web service, Docker, Singapore region	$0. Sleeps after 15 min idle; first request then takes about 1 minute.
Graph database	FalkorDB Cloud Free, AWS ap-south-1 (Mumbai)	$0. No TLS, no backups; the API reloads demo data automatically if the database is empty.
LLMs	Groq free tier, Gemini free tier	$0. Rate limited; the app falls back or uses templates.
Code	Public GitHub repo, MIT license	$0

No payment method is attached to any account, so nothing can be charged; the worst case is a service pausing. Secrets live only in Render environment variables and your local *.env*, which Git ignores.

8. Tracks: what to say for each

Track	Focus points they list	Where Blast Radius shows it
01 Agents That Act on Connected Data (primary)	Multi-hop reasoning; graph queries and algorithms as tools; tool selection through MCP; actions from the graph; explainable results	1-4 hop blast radius and 2-hop change search; PageRank and shortestPath as tools; MCP server with 15 tools; rollback and paging actions; every answer has paths and cited node IDs
02 Agent Memory and Coordination	Episodic, semantic, procedural memory; persistent history; shared state and handoffs; separate memory per tenant	Episode nodes; handoffs via HANDED_OFF_TO read back from the graph; Q&A; history per session; Past sessions reopen without LLM; one graph per tenant
03 Company Brain	Who owns what and who knows what; tracing decisions to source; answers with sources	owners + on-call; fix_history ranks real expertise; decision_trace links ADRs to source documents; postmortems linked to incidents

9. Demo video script (about 4.5 minutes)

Time	Show	Say (in your own words)
0:00-0:30	Live site, idle map	The 2 AM problem: four questions, five tools. Blast Radius answers them from a graph and shows its path.
0:30-1:10	FalkorDB Browser on tenant_acme; the data model in README	Services, deploys, incidents, people and decisions as one graph. One graph per company.
1:10-2:20	Click Run agents; point at map colours, stats and the three agent cards	Triage: 8 at risk, 4 customer-facing, one deploy 38 min before. Investigator: matches an incident 3 weeks ago, 96%. Responder: roll back, page Sam, runbook. About 3 seconds.
2:20-2:50	Open graph evidence / Cypher under an agent	Every claim cites nodes; here is the exact query. The graph decides, the LLM only explains.
2:50-3:30	Refresh; Open under Past sessions; ask two questions	Memory lives in the graph: a new session picks up the incident. The assistant picks tools by itself.
3:30-4:00	Click a different service on the map; switch company to globex	Any service's blast radius on demand. Tenants are fully isolated.
4:00-4:30	README tracks section; repo	Three tracks, free stack, 61 tests that check every query against an independent calculation.

Before recording: open the live site 2 minutes early so Render is awake, click Reset demo, and close other tabs. Record at 1080p, keep the cursor slow, and do one practice run.

10. Question bank (answer out loud first)

About the graph

Why a graph database instead of SQL?

The core questions are variable-depth traversals: everything within 4 hops, everything that changed within 2 hops either way. In SQL that is recursive CTEs or loops in code, and it gets harder as you combine it with deploys, people and incidents. In Cypher it is one pattern, and FalkorDB stores adjacency as sparse matrices, so these traversals are fast enough to run inside every agent step.

Why not just vector search (RAG) over documents?

RAG finds text that looks similar. It cannot tell you that checkout depends on loyalty which depends on postgres-payments, or that the person who fixed the last incident is on call now. Those are relationships, not text. We use the graph for facts and the LLM only to phrase them.

Walk me through the blast radius query.

Start at the alerting service using the name index. Match any path of 1 to 4 DEPENDS_ON edges that ends at it, so every node at the start of such a path depends on it. Return each service, the path length and the node names along the path. Keep the shortest path per service in Python.

Why does recent_deploys look in both directions?

Because the cause is often not in the failing service. In the demo the database did not change; a caller raised its connection pool and exhausted the database. So we look at services that depend on it and services it depends on.

How is one company's data kept separate?

Each tenant is its own named graph in one FalkorDB instance. Every API call selects the graph by validated tenant name, so a query physically cannot match another tenant's nodes. There is a test that loads two tenants and checks this.

What would change with 5,000 services?

The traversals stay bounded by hop limits and start from an index, so they scale with the neighbourhood, not the whole graph. I would add a cap on returned paths, cache PageRank instead of computing it per request, ingest dependencies from real sources such as service mesh or tracing data, and move from FalkorDB Free to a replicated instance.

About the agents and the LLM

How do the agents share state?

Through the graph. Each writes an Episode node linked to the incident, to the nodes it cites and to the next agent. The next agent reads that Episode with a query. No state is passed in memory, so any process or later session sees the same thing.

How do you stop the LLM from making things up?

Three layers. Facts come from Cypher, not the model. The model must cite node IDs, and any ID that was not in the facts makes us reject the summary and use a template. For questions, it can only call fixed tools, and invented IDs are flagged. I also found and fixed a case where the model confused a past cause with a fix, by naming the field unambiguously and telling the Investigator not to recommend actions.

What happens when the LLM is rate limited or down?

The client retries 503s, then falls back from Groq to Gemini. If both fail, the agents still complete with template summaries because none of the decisions depend on the LLM.

How is the confidence computed? Is it real?

It is a transparent heuristic: 35% recency, 25% proximity, 40% history. It is not learned. With real incident data I would fit the weights on past incidents with known causes.

What is MCP and why add it?

The Model Context Protocol lets any compatible AI client discover and call tools. Our MCP server exposes the same graph tools, so Claude Desktop or an IDE agent can investigate incidents using our graph without our UI.

About engineering and process

How do you know the queries are correct?

The tests compute every answer independently with breadth-first search in Python over the same data and compare it with FalkorDB's result. 61 tests run in about 3 seconds locally.

Why synthetic data?

The rules require public, synthetic or own data. The generator is deterministic, which also makes the tests possible. The architecture is hand-written so the demo is realistic.

What did you build and what did the AI assistant build?

Answer honestly: Claude drafted most of the code; I set up and ran every module locally and in the cloud, verified each with tests and real runs, found issues (wrong Git repo, failed LLM keys, the cause-versus-fix error), made the deployment choices and can explain every query. The README discloses this, as Rule 11 requires.

What are the limitations?

Synthetic data; a heuristic confidence score; the free cloud database has no TLS or backups; Render's free tier sleeps; the LLM quotas are small. Each has a clear next step, which is a good thing to say.

11. Commands cheat sheet

```
docker compose up -d                       # FalkorDB on :6379, Browser on :3000
.venv\Scripts\activate
pytest                                     # 61 tests, no LLM calls
python -m blast_radius.cli seed            # reload 3 companies
python -m blast_radius.cli respond         # run the 3 agents on the open alert
python -m blast_radius.cli memory inc-alert-demo-live
python -m blast_radius.cli ask "Who should I talk to about why payments has its own database?"
python -m blast_radius.cli tool blast_radius service=kafka
uvicorn blast_radius.api:app --reload      # UI on http://localhost:8000
python -m blast_radius.mcp_server          # MCP server over stdio
```